

Practical 2:

Aim:

Building a REST API with Flask or FastAPI.

- a. Create a simple REST API using Flask or FastAPI with CRUD operations for a "To-Do List" application.
- b. Test the API using Postman or curl.

Solution:

Step 1: Check if Python is Installed

Open **Command Prompt (CMD)** and run:

```
python --version
```

output:

Python 3.12.4

Step 2: Create a Project Folder

Create a folder for your Flask project:

Flask_Project

Open Command Prompt and navigate to it:

```
cd path\to\Flask_Project
```

Example:

```
cd C:\Users\Nikhita\Documents\Flask_Project
```

Step 3: Create a Virtual Environment (Recommended)

Inside the project folder, run:

```
python -m venv venv
```

This creates a virtual environment named venv.

Activate it:

[Windows](#)

```
venv\Scripts\activate
```

You should see:

(venv) C:\Users\Nikhita\Documents\Flask_Project>

Step 4: Install Flask

With the virtual environment activated, run:

```
pip install flask
```

Step 5: Verify Installation

```
pip show flask
```

or

```
python -m flask --version
```

Example output:

Flask 3.x.x

Werkzeug 3.x.x

Step 6: Create Your First Flask App

Create a file named app.py:

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "Hello, Flask!"
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

Step 7: Run the Flask App

In Command Prompt:

```
python app.py
```

You should see:

* Running on http://127.0.0.1:5000

Open your browser and visit:

http://127.0.0.1:5000

Output:

Hello, Flask!

Code 2:

Replace the code with:

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({
        "message": "Welcome to my first Flask API",
        "status": "success"
    })

if __name__ == '__main__':
    app.run(debug=True)
```

Run again:

```
python app.py
```

Visit:

http://127.0.0.1:5000

Output:

```
{
  "message": "Welcome to my first Flask API",
  "status": "success"
}
```

This is your **first REST API endpoint** in Flask.

Code 3:

```
from flask import Flask, jsonify, request
```

```

app = Flask(__name__)

# Sample data storage
todos = [
    {
        "id": 1,
        "title": "Learn Flask",
        "completed": False
    }
]

# CREATE a new task
@app.route('/todos', methods=['POST'])
def create_todo():
    data = request.get_json()

    new_todo = {
        "id": len(todos) + 1,
        "title": data['title'],
        "completed": False
    }

    todos.append(new_todo)

    return jsonify(new_todo), 201

# READ all tasks
@app.route('/todos', methods=['GET'])
def get_todos():
    return jsonify(todos)

# READ a single task
@app.route('/todos/<int:id>', methods=['GET'])
def get_todo(id):
    todo = next((t for t in todos if t['id'] == id), None)

    if todo is None:
        return jsonify({"message": "Task not found"}), 404

    return jsonify(todo)

# UPDATE a task
@app.route('/todos/<int:id>', methods=['PUT'])
def update_todo(id):
    todo = next((t for t in todos if t['id'] == id), None)

    if todo is None:
        return jsonify({"message": "Task not found"}), 404

    data = request.get_json()

    todo['title'] = data.get('title', todo['title'])
    todo['completed'] = data.get('completed', todo['completed'])

```

```
        return jsonify(todo)

# DELETE a task
@app.route('/todos/<int:id>', methods=['DELETE'])
def delete_todo(id):
    global todos

    todo = next((t for t in todos if t['id'] == id), None)

    if todo is None:
        return jsonify({"message": "Task not found"}), 404

    todos = [t for t in todos if t['id'] != id]

    return jsonify({"message": "Task deleted successfully"})

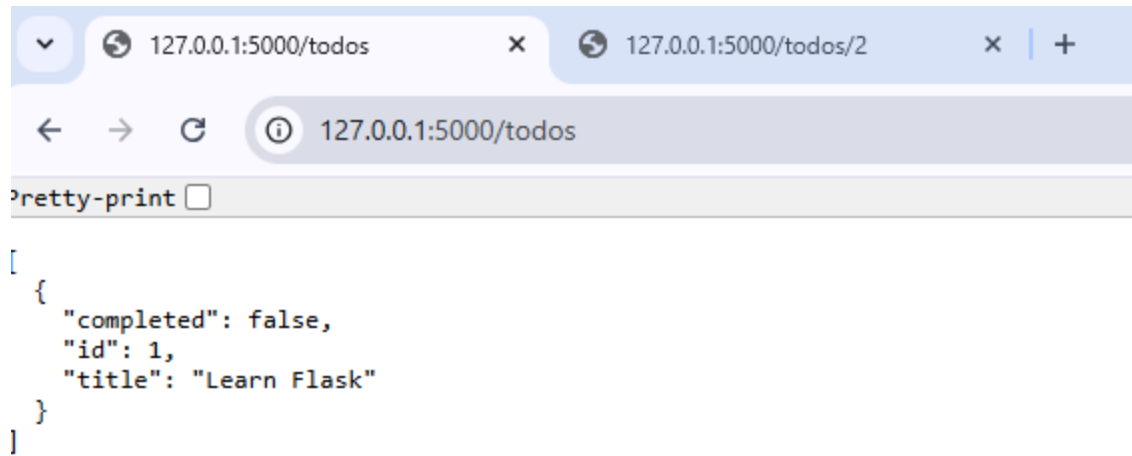
if __name__ == '__main__':
    app.run(debug=True)
```

Run again:

```
python app.py
```

Visit:

<http://127.0.0.1:5000/todos>
<http://127.0.0.1:5000/todos/2>



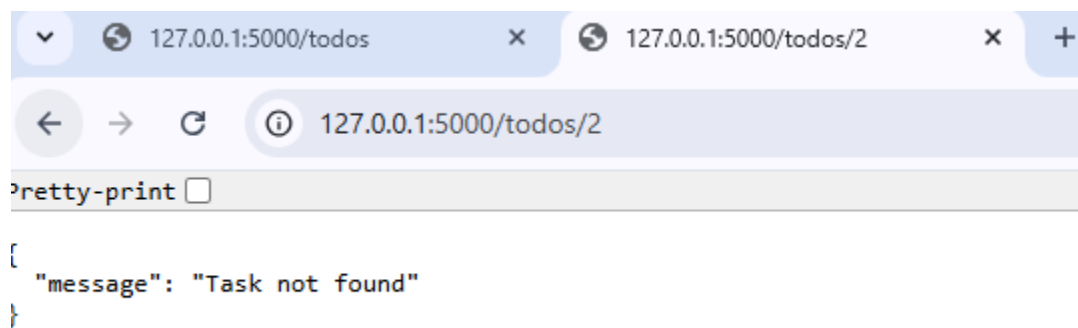
127.0.0.1:5000/todos

127.0.0.1:5000/todos/2

127.0.0.1:5000/todos

pretty-print ☐

```
[
  {
    "completed": false,
    "id": 1,
    "title": "Learn Flask"
  }
]
```



127.0.0.1:5000/todos

127.0.0.1:5000/todos/2

127.0.0.1:5000/todos/2

pretty-print ☐

```
{
  "message": "Task not found"
}
```

```
C:\Users\192>curl http://127.0.0.1:5000/todos
[
  {
    "completed": false,
    "id": 1,
    "title": "Learn Flask"
  }
]

C:\Users\192>curl http://127.0.0.1:5000/todos/1
{
  "completed": false,
  "id": 1,
  "title": "Learn Flask"
}
```

```
C:\Users\192>curl -X DELETE http://127.0.0.1:5000/todos/2
{
  "message": "Task not found"
}
```

```
C:\Users\192>curl -X POST http://127.0.0.1:5000/todos -H "Content-Type: application/json" -d '{"title": "Complete REST API Assignment"}'
{
  "completed": false,
  "id": 2,
  "title": "Complete REST API Assignment"
}
```

1. GET All Tasks

```
curl http://127.0.0.1:5000/todos
```

Output:

```
[
  {
    "id": 1,
    "title": "Learn Flask",
    "completed": false
  }
]
```

2. POST (Create)

```
curl -X POST http://127.0.0.1:5000/todos -H "Content-Type: application/json" -d '{"title":"Complete REST API Assignment"}'
```

Output:

```
{
  "id": 2,
  "title": "Complete REST API Assignment",
  "completed": false
}
```

3. PUT (Update)

```
curl -X PUT http://127.0.0.1:5000/todos/1 -H "Content-Type: application/json" -d '{"title":"Learn Flask API","completed":true}'
```

Output:

```
{
  "id": 1,
  "title": "Learn Flask API",
  "completed": true
}
```

4. DELETE

```
curl -X DELETE http://127.0.0.1:5000/todos/1
```

Output:

```
{
  "message": "Task deleted successfully"
}
```

5. Verify Deletion

```
curl http://127.0.0.1:5000/todos
```

Output:

```
[
  {
    "id": 2,
    "title": "Complete REST API Assignment",
    "completed": false
  }
]
```

Quick Reference for Viva

HTTP Method	Endpoint	Purpose
GET	/todos	Retrieve all tasks
GET	/todos/1	Retrieve a specific task
POST	/todos	Create a new task
PUT	/todos/1	Update an existing task
DELETE	/todos/1	Delete a task

This summary matches the Flask code you provided and is sufficient for demonstrating all CRUD operations using `curl`.